PRACTICAL - 11

Aim: Save/load serialization (Student.java + Driver.java)

Code Screenshot:

Student.java

```java
import java.io.Serializable;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String name;
    private transient String password;

    public Student(int id, String name, String pw) {
        this.id = id; this.name = name; this.password = pw;
    }

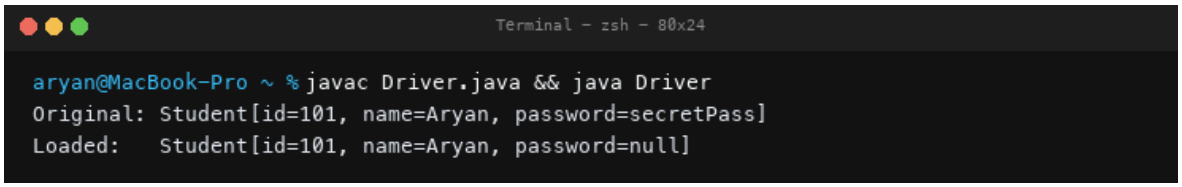
    @Override
    public String toString() {
        return "Student[id=" + id + ", name=" + name + ", password=" + password + "]";
    }
}
```

Driver.java

```java
import java.io.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        File file = new File("students.dat");
        Student s1 = new Student(101, "Aryan", "secretPass");
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(file))) {
            oos.writeObject(s1);
        }
        Student loaded;
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(file))) {
            loaded = (Student) ois.readObject();
        }
        System.out.println("Original: " + s1);
        System.out.println("Loaded:   " + loaded);
        file.delete();
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered Java object persistence using java.io.Serializable and serialVersionUID.
- Managed binary serialization streams with ObjectOutputStream and ObjectInputStream.
- Protected sensitive credentials using the transient modifier.
- Ensured robust I/O leak prevention through try-with-resources.
- Understood JVM byte-stream reconstitution mechanics.

Aim: Log analyzer (LogAnalyzer.java + Driver.java)

Code Screenshot:

LogAnalyzer.java

```java
import java.nio.file.*;
import java.util.List;

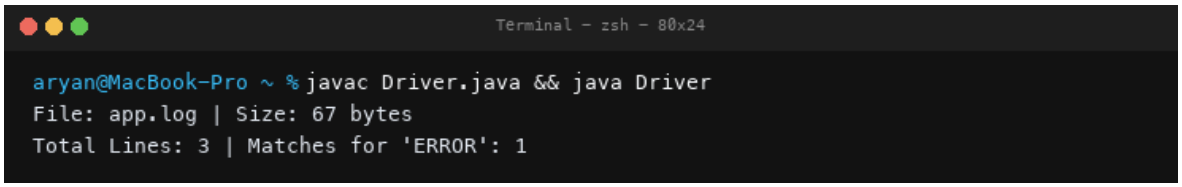
public class LogAnalyzer {
    public static void analyze(Path file, String keyword) throws Exception {
        List<String> lines = Files.readAllLines(file);
        long matches = lines.stream().filter(l -> l.contains(keyword)).count();
        long size = Files.size(file);
        System.out.println("File: " + file.getFileName() + " | Size: " + size + " bytes");
        System.out.println("Total Lines: " + lines.size() + " | Matches for '" + keyword + "': " + matches);
    }
}
```

Driver.java

```java
import java.nio.file.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        Path log = Paths.get("app.log");
        Files.writeString(log, "INFO System started\nERROR Database timeout\nINFO Retrying connection\n");
        LogAnalyzer.analyze(log, "ERROR");
        Files.deleteIfExists(log);
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modern Java NIO.2 file handling with Path, Paths, and Files.
- Inspected OS file attributes including file size using Files.size().
- Streamed and filtered text lines efficiently with Java Collections and Streams.
- Implemented high-performance file content search pipelines.
- Applied safe temporary resource creation and deletion operations.